

Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses

一句话总结：本文提出 AHE（Agentic Harness Engineering），一个可观测性驱动的闭环系统，通过另一个 Agent（Evolve Agent）自动演化编码 Agent 的 harness（系统提示词、工具、中间件、长期记忆等模型外部工程组件），10 轮迭代将 Terminal-Bench 2 pass@1 从 69.7% 提升至 77.0%，超越人工设计的 Codex-CLI 和自动演化基线 ACE/TF-GRPO，且冻结 harness 无需重新演化即可跨 benchmark 和跨模型家族迁移。

1 Intro

1.1 Motivation

编码 Agent 的性能不仅取决于基座模型，还取决于周围的 **harness**（脚手架）—— system prompt、tools、middleware、skills、long-term memory 等模型外部可编辑工程组件。harness 设计对长时域编码 benchmark 的性能有**实质性影响**，相同模型 + 不同 harness = 显著不同的 pass rate。

但存在一个**适配缺口**：最优 harness 是模型特定的（model-specific），而基座模型在快速迭代（GPT-5.4、DeepSeek-V4、Qwen-3.6 每月更新），手工调整根本跟不上节奏。

已有的自动化方法（ACE、TF-GRPO 等）主要优化 prompt，**无法触及** tools、middleware、memory 这些更深层的组件。

1.2 核心主张

harness 优化的瓶颈是可观测性（observability），而非 Agent 能力。一旦演化 Agent 获得结构化上下文和清晰的动作空间，就能可靠收敛到更好的 harness 设计。

三个"看不见"的问题：

- 1. 看不见组件边界——哪些可以改？改了影响什么？
- 2. 看不见轨迹信号——百万 token 日志中真正有用的信号在哪？
- 3. 看不见编辑后果——这个改动帮了什么、坏了什么？

1.3 贡献

- 1. **AHE 框架**：三个可观测性支柱 + 文件级可证伪契约
- 2. **实验验证**：10 轮 69.7%→77.0%，超越人工和自动基线，冻结 harness 可迁移
- 3. **分析发现**：组件交互非加性 + 自归因对修复可靠但对回退盲

2 Method

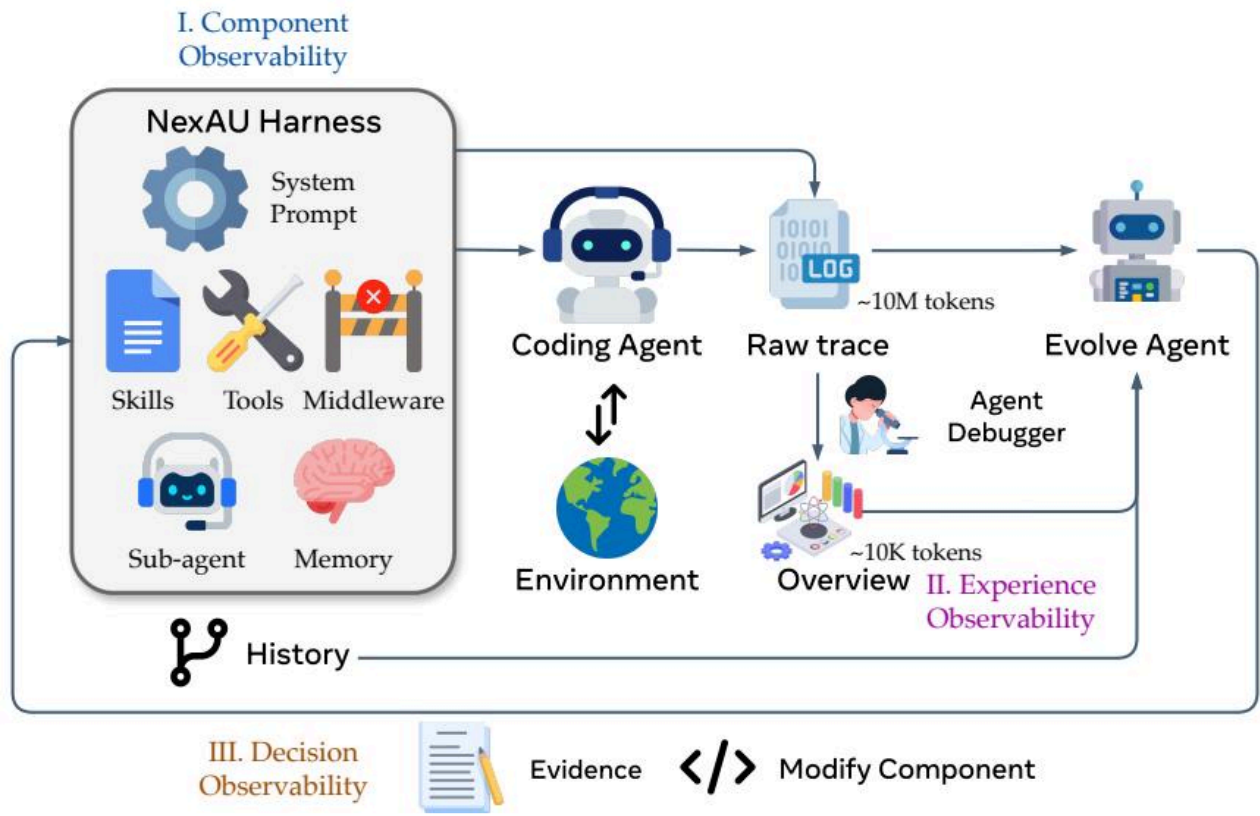


Figure 2: AHE 流水线总览——三个可观测性表面（组件/经验/决策）串联成一个闭环，每个编辑变成下一轮可验证的可证伪预测。

2.1 支柱 1：组件可观测性（Component Observability）

核心思想：把 harness 的每个组件变成一个独立的文件，使故障模式→组件类的映射变成故障模式→文件的映射。

通过 NexAU 框架实现。NexAU 是一个通用 Agent 框架（[GitHub](#)），将 harness 的 7 种可编辑组件暴露为固定路径的独立文件：

```
workspace/
├── systemprompt.md          # 系统提示词（定义工作风格、行为规则）
├── LongTermMEMORY.md       # 长期记忆（跨会话积累的经验教训）
├── code_agent.yaml         # Agent 配置（工具/中间件/技能注册）
├── tool_descriptions/      # 工具描述（YAML schema，模型读取以理解工具）
├── tools/                  # 工具实现（Python 函数，直接控制工具行为）
├── middleware/             # 中间件（Python 类，hook 到 agent 循环管线）
├── skills/                 # 技能包（SKILL.md + 配置，按需加载的可复用能力）
└── sub_agents/             # 子 Agent 配置（委派子任务的独立 agent）
```

组件间的松耦合设计是关键——添加一个 middleware 不需要编辑 system prompt，添加一个 skill 不需要修改任何 tool。每个组件独立可测、独立可回滚。

种子 harness 的最小化策略：初始配置 NexAU。故意只保留一个 `run_shell_command` 工具，没有 middleware、没有 skills、没有子 agent、没有长期记忆。这是为了防止“种子 already fitted to the target benchmark”导致后续每个编辑的 attribution 被污染——如果种子已经很好了，我们无法区分增益来自演化循环还是种子本身。最小种子迫使每个新增组件必须通过实测 rollouts 证明自己的价值。

文件级 git 追踪带来免费的审计能力：每个逻辑编辑 = 一个 git commit，天然拥有 diff 和 rollback 粒度。这比传统 prompt engineering 中散落在数百行非结构化文本里的修改要清晰得多。

2.2 支柱 2：经验可观测性（Experience Observability）

核心思想：把百万 token 的原始轨迹蒸馏为演化 Agent 能消化的分层证据语料。

一个 Terminal-Bench 2 任务的单次 rollout 可能产生数十万 token 的 shell 输出、文件内容、模型推理链。原始日志虽然完整，但信号被淹没在噪声中。Agent Debugger 的作用是充当信号提取器。

🔗 Agent Debugger 源码分析

Agent Debugger 不是传统的程序化分析器，而是一个**NexAU Agent** (`debugger_agent`)，嵌入在 `agents/evolve_agent/skills/agent-debugger-cli/` 中。通过查看源码发现其关键设计：

- **硬限制 20 次工具调用** (`system_prompt.md` 中定义："You have a hard budget of 20 tool-calling iterations")
- **工具集**： `read_file` (带 `offset/limit` 读大文件)、 `search_file_content` (正则搜索)、 `glob`、 `run_shell_command`、 `complete_task`
- **5 阶段工作流**：
 1. **Skim** (iter 1–3)： `read_file` 看 `trace` 头部，了解消息结构
 2. **Locate** (iter 4–10)： `search_file_content` 正则搜索错误关键词、工具名
 3. **Read in context** (iter 11–15)：读命中位置的上下文，建立因果链
 4. **Cross-trace diff** (iter 16–18)：多 `trace` 对比，找分歧点
 5. **Finalize** (iter ≤20)： `complete_task` 输出结构化 JSON
- **输出 5 种 issue 类型**： {工具错误, 幻觉, 循环, 不合规, 截断}
- **两种模式**： `ask` (自由文本回答) / `check` (结构化 issues 列表)
- **trace 格式支持**： `openai_messages` (默认)、 `langfuse`、 `in_memory_tracer`
- **中间件**： `LongToolOutputMiddleware` (截断 >10K 字符输出) + `ContextCompactionMiddleware` (阈值 0.75, 保留最近 10 轮)
- **runner.py**：3 次指数退避重试、预算溢出回退 (输出 `[budget-exceeded]`)、`check` 模式输出 `schema` 验证失败自动重试一次

两层报告结构

Agent Debugger 输出**两层报告**：

1. **Per-task analysis**：每个任务的 `pass/fail` 状态 + 根因分析 (逐任务文件存储在 `analysis/detail/{task_name}.md`)
2. **Benchmark-level overview**：聚合所有报告的单文档入口 (`analysis/overview.md`)，是 `Evolve Agent` 每轮首先读取的文件

原始轨迹保留供验证，支持**渐进式信息披露** (progressive disclosure) ——`Evolve Agent` 默认读摘要，需要时再钻取原始 `trace` 文件。

2.3 支柱 3：决策可观测性 (Decision Observability)

核心思想：**每次编辑都变成一个可证伪的契约**，用下一轮任务结果自动验证。

通过 **Evolve Agent + Change Manifest** 实现。`Evolve Agent` 做的每条编辑必须附带一份 `Change Manifest`：

```
{
  "id": "chg-1",
  "type": "new|improvement|rollback",
  "description": "What was changed and why",
  "files": ["relative/to/workspace/file.py"],
  "failure_pattern": "The failure class this addresses",
  "predicted_fixes": ["task-name-a", "task-name-b"],
  "risk_tasks": ["task-name-c"],
  "constraint_level": "middleware|tool_impl|tool_desc|skill|prompt",
  "why_this_component": "Why this component level was chosen"
}
```

自动验证与回滚 (Attribution 阶段)

下一轮运行时，Algorithm 1 的 Phase 3 (Attribute) 执行以下流程：

1. 读取上轮的 Change Manifest
2. 将 `predicted_fixes` 与本轮观察到的 task-level pass/fail delta 交叉比对
3. 将 `risk_tasks` 与本轮实际回归的任务交叉比对
4. 生成 per-edit verdict：每个编辑被标记为 **KEEP** (预测命中)、**IMPROVE** (方向正确但不够)、或 **ROLLBACK + PIVOT** (预测失败，回滚并换组件层级重试)

验证失败的编辑在文件粒度回滚——因为每个编辑是一个 git commit，revert 就是 `git revert`。

两个约束保证可控性

1. **Controllability**: Evolve Agent 只能写 `workspace/` 目录。`runs/` 只读、LLM 配置不可修改、tracer/verifier 不可触碰、种子 system prompt 标记为不可删除。这防止了演化 Agent 走捷径 (如关闭验证器、换模型、提高推理预算)。
2. **Evidence-driven**: 每条编辑必须引用具体的失败证据、推断的根因、针对性的修复方案和预测影响。不允许基于"直觉"或"最佳实践"的改动。

2.4 AHE 外循环 (Algorithm 1)

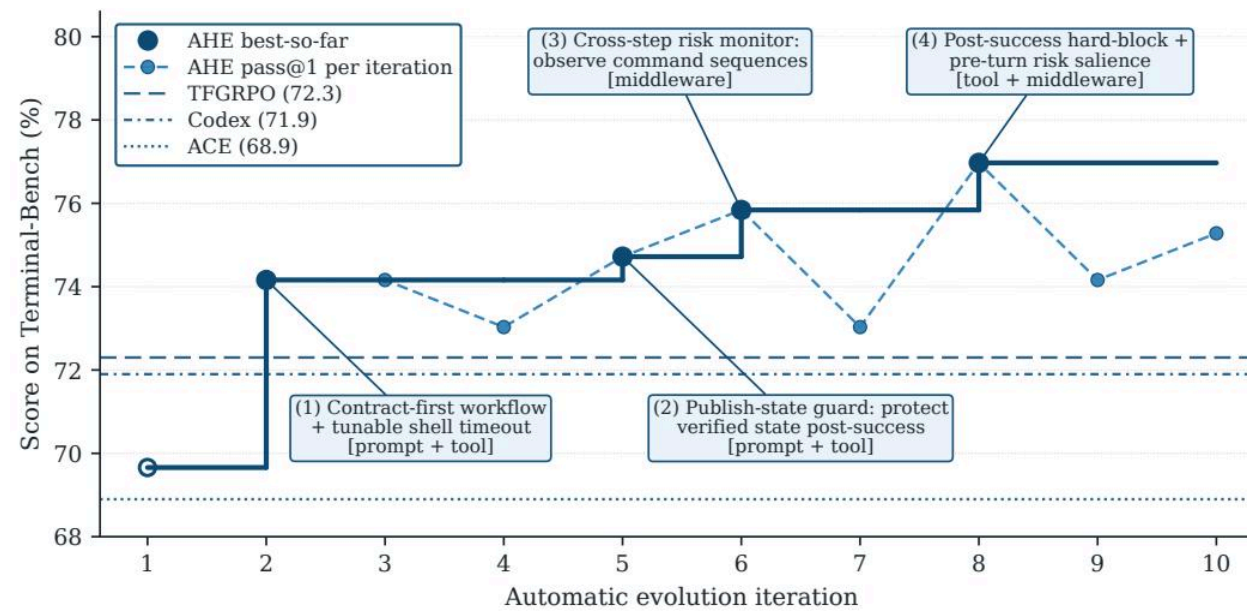


Figure 1: AHE 在 Terminal-Bench 2 上的演化曲线。三个角色 Agent 共享同一基座模型，增益完全来自 harness 编辑。

算法 1: AHE 外循环

输入：种子 harness H_0 ，基座模型 M ，benchmark D ，每任务 rollout 数 k ，最大迭代数 N

$H_{\text{best}} \leftarrow H_0$

for $t = 1$ to N :

Phase 1: $T_t \leftarrow \text{ROLLOUT}(M, H_{t-1}, D, k)$

k 次 rollout/任务

Phase 2: $T_t \leftarrow \text{CLEAN}(T_t)$

去 base64、去重工具输出

具输出

Phase 3: if $t \geq 2$:

$V_t \leftarrow \text{ATTRIBUTE}(C_{t-1}, T_{t-1}, T_t)$

验证上轮 manifest

$H_{t-1} \leftarrow \text{ROLLBACK}(H_{t-1}, V_t)$

回滚失败编辑

Phase 4: $R_t \leftarrow \text{AGENTDEBUGGER}(T_t)$

分层蒸馏轨迹

Phase 5: $(H_t, C_t) \leftarrow \text{EVOLVE}(H_{t-1}, R_t, V_t)$

编辑 + 新 manifest

Phase 6: $\text{COMMIT}(H_t, C_t, t)$

git commit 标记

迭代

if $\text{PASS@1}(T_t) > \text{PASS@1}(H_{\text{best}})$: $H_{\text{best}} \leftarrow H_t$

```
return H_best
```

关键设计选择：

1. **$k \geq 2$ rollouts/任务**：使每个任务携带 pass-rate 信号（0/2、1/2、2/2）。partial-pass 任务（1/2）是最有价值的诊断信号——对比同一个任务的通过和失败 rollout，可以精确定位分歧点。
2. **Attribution 在蒸馏之前运行**：上轮的 manifest 先验证，验证结果作为证据进入 Agent Debugger 的蒸馏语料。这使得 manifest 不仅是"自说自话的理由"，而是"被数据验证过的契约"。
3. **首轮并行 seed skills**：一个单 shot explore agent 并行运行，从 NexAU 源码和公开编码 Agent 参考中提取少量可复用 skills。这些 skills 从第 2 轮起不享有特殊保护——Evolve Agent 可以保留、修改或删除它们。
4. **每轮两个角色 Agent 协作**：
 - **Agent Debugger**（Phase 4）：分析轨迹，输出结构化根因报告
 - **Evolve Agent**（Phase 5）：读报告，做编辑，写 manifest
 - 两个角色共享同一基座模型（实验中均为 GPT-5.4 high reasoning），隔离了增益来源——如果 Debugger 用了更强的模型，我们无法区分增益来自更好的分析还是更好的编辑
5. **Git 作为版本控制**：每次迭代一个 commit tag，完整保留演化历史。可以随时 `git checkout` 到任意历史版本进行对比。

3 实验结果

3.1 主实验（RQ1）

在 Terminal-Bench 2（89 个任务）上，10 轮 AHE 演化，约 32 小时完成：

方法	总体 (89)	Easy (4)	Medium (55)	Hard (30)
opencode	47.2%	75.0%	52.7%	33.3%
terminus-2	62.9%	75.0%	74.5%	40.0%
Codex-CLI	71.9%	75.0%	80.0%	56.7%
NexAU ₀ 种子	69.7%	87.5%	78.2%	51.7%
ACE	68.9%	91.7%	78.2%	48.9%
TF-GRPO	72.3%	100.0%	79.4%	55.6%
AHE	77.0%	100.0%	88.2%	53.3%

AHE 超越所有人工设计和自动演化基线。唯一例外是 Hard 上略低于 Codex-CLI，但消融表明这是组件间非加性干扰。

3.2 跨 Benchmark 迁移（SWE-bench-verified）

冻结 AHE harness，不做任何重新演化：

指标	NexAU ₀	ACE	TF-GRPO	AHE
成功率	75.2%	74.6%	74.2%	75.6%
Tokens k	526	679	582	461

AHE 综合成功率最高，且比种子省 12% token，比 ACE 省 32%。ACE 和 TF-GRPO 反而低于未修改种子——它们学到的 prompt 策略在新任务上变成纯额外开销。

3.3 跨模型迁移

在 5 个不同模型上直接复用 AHE harness（不做重新演化）：

模型	增益
deepseek-v4-flash	+10.1 pp
qwen-3.6-plus	+6.3 pp
gemini-3.1-flash-lite	+5.1 pp
GPT-5.4 medium	+2.3 pp
GPT-5.4 xhigh	+2.3 pp

跨家族增益 > 家族内增益。越远离饱和的模型获益越大。

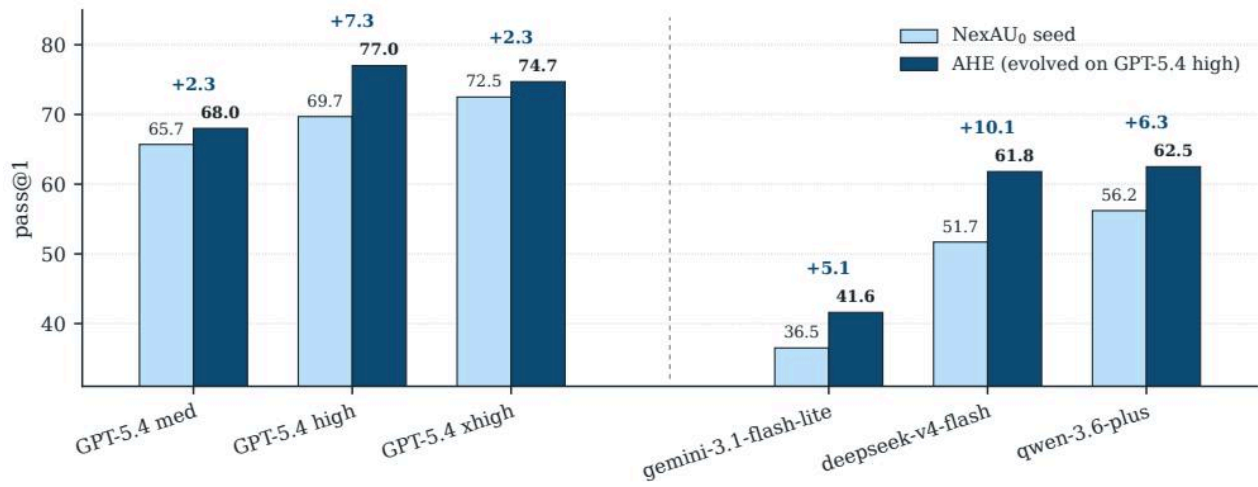


Figure 3: 跨模型迁移。在 GPT-5.4 high 上演化的 AHE workspace 直接在 5 个替代模型上评估，全部正增益，跨家族增益（DeepSeek/Qwen/Gemini）大于家族内增益。

3.4 组件消融

把 AHE 演化的每个组件单独替换回种子：

单独替换	Δ pass@1
+ AHE memory only	+5.6 pp
+ AHE tool only	+3.3 pp
+ AHE middleware only	+2.2 pp
+ AHE system_prompt only	-2.3 pp

⚠ 反直觉发现

单独使用 AHE 演化的 system prompt 反而**降低性能**。原因：prompt 中的规则（如"结束前验证结果"）需要 middleware 的 finish-hook 强制执行、需要 memory 中的具体验证模式才能生效。脱离结构化组件，prompt 只是散文。

结论：事实性 harness 结构可迁移，散文级策略不可迁移。

三个正组件单独增益之和 +11.1 pp > 全量 +7.3 pp，因为组件趋向同一种行为模式（闭合验证），堆叠时产生冗余检查。

3.5 自归因分析

维度	精度	召回	vs 随机基线
修复预测	33.7%	51.4%	~5x
回归预测	11.8%	11.1%	~2x

9 轮中 Agent 发出 43 个回归预测仅 5 个命中，40 个实际回归被完全漏判。

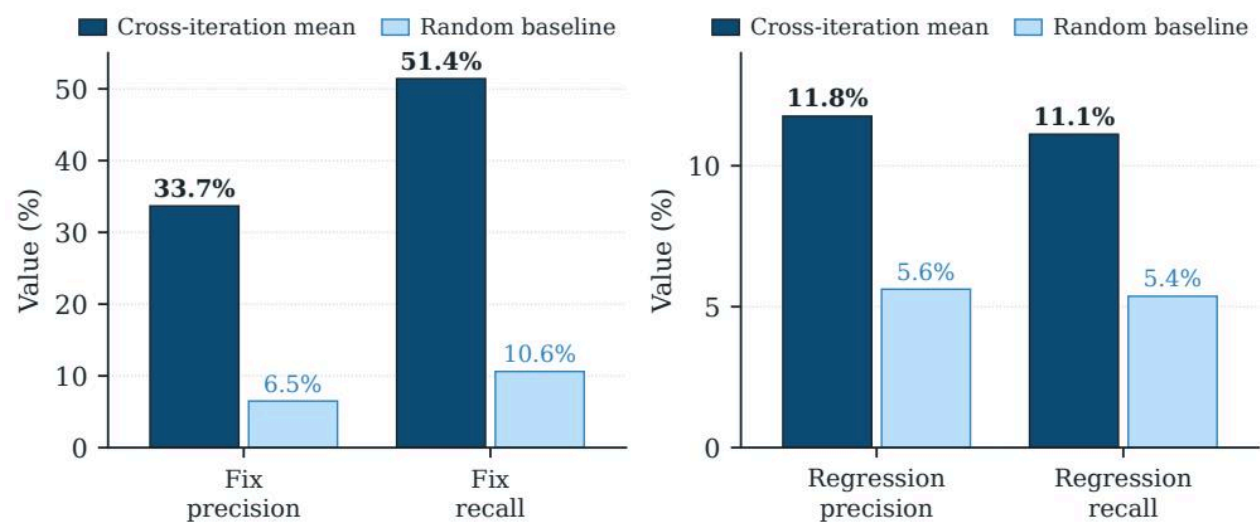


Figure 4: 自归因分析的 9 轮精度和召回。左：修复预测 (~5x 随机基线)。右：回归预测 (~2x 随机基线)。Agent 能解释“为什么有帮助”，但无法预测“会破坏什么”。

4 案例研究：AHE 如何实际工作？

论文附录给出 4 个失败→修复轨迹，揭示层级递进演化路径：

轨迹 1: db-wal-recovery (迭代 2, prompt 层)

Agent 从损坏 WAL 文件重建数据库，但用猜测模式编造缺失行。→ system prompt 追加 8 条通用规则（契约优先、评估者镜像等），0/2→2/2。

Repo	<i>N</i>	ACE	TF-GRPO	NexAU ₀	AHE
All	500	1.10	1.27	1.43	1.64
django	231	1.12	1.35	1.50	1.67
sympy	75	1.15	1.19	1.43	1.48
sphinx-doc	44	0.62	0.78	0.93	1.07
matplotlib	34	1.14	1.33	1.51	1.88
scikit-learn	32	2.08	2.48	3.06	3.40
pydata	22	1.37	1.50	2.00	2.15
astropy	22	1.08	1.26	0.82	1.81

Figure 5: 轨迹 1 前后对比。左列：失败 rollout（从缓存 stdout 恢复字节→猜测缺失行→自创代理检查）。中列：chg-1 规则如何逐缺口覆盖。右列：通过 rollout（重新读取契约→原始磁盘恢复→镜像验证器）。

轨迹 2: path-tracing (迭代 5, prompt + tool 层)

Agent 渲染正确图像，但 `rm -rf` 删掉了输出文件。→ prompt 规则 + shell 工具安装发布状态守卫（硬阻塞删除已验证输出），0/2→2/2。

轨迹 3: mcmc-sampling-stan (迭代 6, tool + middleware 层)

Agent 用网格积分代替 MCMC、后台启动采样但未完成就杀掉。→ 工具层保护脚本入口点 + ExecutionRiskHintsMiddleware 监控 7 种跨步风险模式，连续 5 轮失败后

0/2→2/2。

轨迹 4: configure-git-webserver (迭代 8, 硬化 + 提升)

Agent 用 override token 绕过保护执行破坏性清理。→ shell 守卫升级为硬阻塞 + 中间件风险提示提升到模型推理上下文 (FRAMEWORK 提醒), 0/2→2/2。

🔗 演化路径总结

prompt (快速低成本) → tool (硬执行保障) → middleware (深层行为监控) → 硬化 + 提升

从软约束到硬约束的自然递进。

5 结论

AHE 将 harness 设计从"手工工艺"升级为"可自动化的学习问题"。核心创新在于通过三个可观测性支柱解决了 attribution 瓶颈——组件可见、轨迹可钻取、决策可证伪。实验表明, 10 轮演化可产生超越人工设计的 harness, 且冻结后可跨 benchmark 和模型家族迁移。

6 思考

6.1 优点

- 1. **范式洞察的正确性**: "harness 优化的瓶颈是可观测性而非 agent 能力"这个论点有深刻的方法论意义。大多数 agent 研究关注"让 agent 更强", AHE 关注"让 agent 看到自己在做什么"——三个支柱各自解决一种"看不见"的问题。
- 2. **可证伪契约的精巧设计**: Change Manifest 把每个编辑变成可测量的契约, 用下一轮任务结果自动验证。这比 Reflexion 的自然语言反思或 ACE 的上下文 playbook 都更严格——从"自说自话"变成了"用数据说话"。
- 3. **组件消融的反直觉发现**: system prompt 单独不工作、三个正组件之和 > 全量组合。这些发现对所有 coding agent 开发者都有指导价值——别指望 prompt 写得好就行, 结构化组件 (tools/middleware/memory) 才是可迁移的核心。
- 4. **案例研究的深度**: 4 个轨迹的前因后果交代得非常清楚, 特别是从 prompt→tool→middleware 的层级递进, 展示了演化 agent 如何从简单策略自然地走向复杂策略。
- 5. **代码开源**: 完整实现可在 GitHub 获取, 包括 NexAU harness 框架、Agent Debugger CLI、Evolve Agent prompt 和所有演化配置。

6.2 缺点与待解决问题

- 1. **对 Terminal-Bench 的深度依赖**: Agent Debugger 的 5 阶段 workflow (Skim→Locate→Read→Diff→Finalize) 高度依赖 terminal trace 的结构化程度——精确的 exit code、stack trace、结构化消息序列。如果 trace 不是这种格式 (如网页浏览、异步操作、主观评估), Agent Debugger 的每个阶段都会退化。SWE-bench 迁移本质上只是"同域不同数据集" (都是代码 + 确定性验证器), 不是真正的跨域迁移。
- 2. **Agent Debugger 的"智能"被 terminal trace 承载**: 通过查看源码, Agent Debugger 本身是一个受 20 次迭代严格限制的 NexAU Agent, 靠 system prompt 中的 5 阶段 workflow 引导。它的分析能力很大程度上来自 trace 的结构化程度, 而非自身的推理能力。
- 3. **回归盲区的系统性问题**: Agent 能可靠预测修复 (~5x 随机), 但几乎无法预测回归 (~2x 随机)。这不是工程疏忽而是根本性不对称——证明有帮助只需找到修复的 case, 预测破坏需要全局推理。这限制了 AHE 在高风险场景中的可信度。

- 4. **组件交互的非加性**：三个正组件之和 +11.1 pp > 全量 +7.3 pp，memory-only 甚至超过全量 AHE。这说明演化 agent 收敛到了一个对 Medium 任务最优但对 Hard 任务次优的权衡，缺乏交互感知的演化策略。
- 5. **操作点耦合**：超时和步数预算在 GPT-5.4 high 上拟合，跨推理层级增益非单调（medium +2.3, high +7.3, xhigh +2.3）。这暗示 harness 演化的价值可能存在一个"能力窗口"——模型太弱时 harness 救不了，模型太强时可有可无。

6.3 "能力窗口"假说

跨模型迁移的增益模式暗示：

- **弱模型获益最大**（deepseek-v4-flash +10.1 pp）
- **强模型获益较小**（GPT-5.4 +2.3 pp）

可能存在一个 **harness 价值的能力窗口**：模型太弱→harness 救不了根本性推理不足；模型太强→能从 prompt 自行推导等效协调模式。**中间地带**（当前 2025-2026 年模型能力快速增长但尚未饱和）恰好是 harness 工程的主战场。

6.4 Harness vs RL 训练：两个正交的优化轴

维度	RL 训练（模型内）	Harness 演化（模型外）
优化目标	模型权重	工程组件（文件）
可审计性	低（黑箱）	高（git commit）
可回滚性	困难	简单（文件级 revert）
跨模型迁移	不适用	部分可迁移
优化成本	极高（GPU 集群）	中等（API 调用）

两者互补：RL 训练提升"原始推理能力"，harness 演化优化"如何将推理能力转化为任务完成"。

6.5 对图像文本编辑任务的启示

我们的任务是**图像中的文本编辑**——在扩散模型生成/编辑的图像中精确控制文字内容。这个任务与 AHE 研究的 coding agent 场景看似相距甚远，但 AHE 的核心方法论有三个可借鉴之处：

6.5.1 可观测性思维的迁移

AHE 的核心洞察——"瓶颈是可观测性而非模型能力"——在图像文本编辑中同样成立。当前文本编辑的主要失败模式包括：字符渲染错误（特别是中文）、文字位置偏移、字体风格不一致、背景破坏。

AHE 的启示是：与其盲目调参，不如先建立**可观测性基础设施**——系统性地记录每个编辑任务的失败模式、根因分类和修复效果，形成可追溯的改进闭环。

6.5.2 可证伪契约在评估中的应用

AHE 的 Change Manifest 思想可以迁移到我们的评估流程中。当我们修改harness

- **predicted_fixes**：这个改动预期修复哪些类型的文字错误？（笔画缺失？位置偏移？字体不一致？）
- **risk_tasks**：这个改动可能破坏哪些之前正常的 case？（长文本？小字号？特殊字符？）
- **per-edit verdict**：下一轮评估后，预测是否命中？

这种"每个改动都是可证伪契约"的思维方式，比"调参→看指标→觉得还行"要严格得多。

6.6 与已有 Wiki 的连接

- 关联概念：[Wiki/Concepts/Agentic Harness Engineering \(AHE\)](#)、[Wiki/Concepts/Harness 编码 Agent 线束](#)、[Wiki/Concepts/Self-Evolving Agents 自演化智能体](#)、[Wiki/Concepts/LLM Agent 架构](#)
- 关联实体：[Wiki/Sources/AHE Agentic Harness Engineering](#)
- 关联主题：[Wiki/Topics/LLM Agent 与工具使用](#)
- 关联来源：[AI阅读笔记/GLM-5 从 Vibe Coding 到 Agentic Engineering](#)（同一范式转变的不同切入角度）